

Introduction to lesson 6 – Practicing Coding and Building Thinking Skills

At this stage, you already have an idea of what coding is, but translating that into working programs can feel challenging and sometimes even a little frustrating. This is completely normal — this phase, where you're learning to persist, try, fail, and try again, is the foundation of all your future skills as a developer. Real learning happens here, through patience, repetition, and active engagement.

I highly recommend practicing exercises like the ones on [Edabit](#)

. Even small, interactive challenges help build the habit of thinking like a programmer and improving problem-solving skills. Beyond coding, activities like playing chess, strategy games, sports with tactics, or even card games like War can strengthen critical thinking and decision-making. These are far more beneficial than fully passive activities — especially short, endless scrolling on TikTok or reels — which train your brain to consume without thinking.

During these exercises, try to solve the problems yourself without relying on ChatGPT or other AI assistants. The goal is to develop your own logic, reasoning, and confidence in writing small functions and programs. Remember: persistence now will pay off massively later, and even small daily practice builds the foundation for a strong problem-solving mindset.

Algorithm = 3 steps

1. Understand input
2. Decide processing steps
3. Determine output

Exercise 1 — alone :

Sum of numbers in a list.

Exercise 2 — Hints Then Code

Return the index of the maximum number in a list.

Hints:

- “We need to track best value so far”
- “We also track its position”
- “Update when bigger found”

Exercise 3 — Alone

Check if a number exists in a list. Return true/false.

Exercise with hint alone 1 — Simple

Sum of positive numbers

Problem: Write a function that takes a list of integers and returns the sum of only the positive numbers (strictly > 0). If there are no positive numbers return 0 .

Example:

Input: $[4, -1, 0, 7, -3]$ → Output: 11 ($4 + 7$)

Hint: loop through the list, add only if $x > 0$.

Exercise with hint alone 2 — Intermediate

Indices of maximum value

Problem: Write a function that returns a list of all indices where the maximum value appears. If the list is empty return an empty list.

Example:

Input: $[2, 5, 3, 5, 1]$ → Output: $[1, 3]$ (5 is max at positions 1 and 3)

Hint: first find the max, then collect indices where $\text{element} == \text{max}$.

Exercise with hint alone 3 — Intermediate

Merge two sorted lists (no built-in sort)

Problem: Given two lists sorted in ascending order, return a single sorted list that merges them. Do **not** use the language builtin sort function; use the two-pointer merging approach.

Example:

Input: $[1, 3, 5]$ and $[2, 4, 6, 7]$ → Output: $[1, 2, 3, 4, 5, 6, 7]$

Hint: keep two indices (i, j), compare elements, append smaller, advance pointer; when one list finishes append the rest of the other

Exercise with hint alone 4 — Debugging (fix logic)

Check sorted (buggy code)

Below is a Python function intended to return `True` if a list is sorted in ascending order and `False` otherwise. It contains a logic bug. **find and fix** the bug and explain why the original failed.

```
def is_sorted(lst):
    for i in range(len(lst)):
        if lst[i] < lst[i+1]:{
            return true
        }
    return false
```

What to do:

- Explain what is wrong.
- Provide corrected code.
- Give at least two test runs showing the bug fixed.

Hints:

- Think about the loop range and the comparison direction.
- What happens when `i` reaches the last index?

Exercise with hint alone 5 — Debugging (fix logic)

Count occurrences (buggy code)

Student wrote this function to count how many times `target` appears in `lst`, but it sometimes returns `0` even when there are occurrences.

```
def count_occurrences(lst, target):
    count = 0
    for i in range(len(lst)):
        if lst[i] == target:
            count = count + 1
        else:
            count = 0
    return count
```

What to do:

- Identify the bug and explain what causes it.
- Fix the code and show corrected output for the test cases.

Hints:

- Consider what happens when a non-target value appears after some matches.
- The count should accumulate across the whole list, not reset.

★ 4. Introduce Very Small Pattern Templates

3 skeletons to reuse:

✓ Loop template

```
for each element in list:  
    ...
```

✓ Count template

```
count = 0  
for item in list:  
    if condition:  
        count += 1  
return count
```

✓ Track best value template

```
best = list[0]  
for item in list:  
    if item > best:  
        best = item
```

These build **pattern thinking** — the key to coding fluency.

also doing some iq test seems like a game for me its motivating and challenging i like it and it help built critical thinking i keep trying to get more score i find it interesting entertaining and helpful as well i suggest to try it
<https://zoneoftools.com/tools/iq-test>